CST1510 Programming for data communication and networks

Resit Coursework 1 - PORTFOLIO - worth 50%

This resit coursework requires you to demonstrate your basic Python skills. You will have a set of tasks to complete each week and need to answer some questions. Each week consists of three programming challenges, gradually increasing in difficulty, followed by questions to showcase your understanding.

Week 1: Getting Started with Python (Variables, Input/Output)

Task 1 (Simple) - Personalized Chatbot

Create a simple chatbot that asks the user their **name**, **favorite hobby**, and **dream job**. The chatbot then responds with a fun message.
Use formatted strings for output.

Example Input:

```
Enter your name: Alex
What is your favorite hobby? Gaming
What is your dream job? Game Developer
```

Desired Output:

```
Hello Alex! It's awesome that you love Gaming.
Becoming a Game Developer sounds exciting. Keep chasing
your dreams!
```

Task 2 (Intermediate) - Fun Temperature Converter

Write a program that converts a **temperature value** between Fahrenheit and Celsius.
Let the user choose the conversion direction.

Example Input:

```
Choose conversion:
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius
Enter choice: 1
Enter temperature: 25
```

Desired Output:

```
25°C is equal to 77°F
```

Task 3 (Advanced) - Superhero Power Calculator

Imagine a game where each superhero's power is based on **their strength (S), speed (P), and intelligence (I)**.
Create a program that asks for these three values and **calculates a hero's overall power level** using the formula:

```
Power Level = (S ** 2) + (P * 2) + (I * 3)
```

Example Input:

```
Enter strength: 5
Enter speed: 10
Enter intelligence: 7
```

Desired Output:

```
Your superhero's power level is: 64
```

QUESTIONS WEEK 1

- Explain how you used variables in your programs this week. Why are variables important in programming?
- Why is it necessary to convert user input from `input()` when performing calculations? Provide an example from your work.
- What are f-strings in Python? How do they improve output formatting compared to string concatenation?
- If you had to improve your chatbot program, what additional functionality would you add, and how would you implement it?
- Did you face any syntax errors? If so, describe one and how you fixed it.

Week 2: Control Structures (Selection & Iteration)

Task 1 (Simple) - Odd or Even Game

Create a program that asks the user for a number and tells them if it's **odd or even**.

Example Input:

```
Enter a number: 13
```

Desired Output:

```
13 is an odd number.
```

Task 2 (Intermediate) - Number Guessing Game 🎯

Make a number-guessing game where the program randomly selects a number between **1 and 100**, and the user has to guess it.
Provide hints (Too high/Too low) until they guess correctly.

Example Interaction:

```
Guess the number: 50
Too high! Try again: 25
Too low! Try again: 37
Correct! The number was 37.
```

Task 3 (Advanced) - Multiplication Battle

Create a **multiplication quiz** where the player must answer **5 random multiplication questions**. The program tracks their score.

Example Run:

```
What is 6 x 7? 42
Correct!
What is 3 x 9? 27
Correct!
What is 8 x 5? 40
Correct!
What is 4 x 6? 23
Wrong! The correct answer was 24.
What is 9 x 9? 81
Correct!

Final Score: 4/5
```

QUESTIONS WEEK 2

- How do `if-elif-else` statements help control the flow of a program? Provide an example from your code.
- Describe the difference between a `for` loop and a `while` loop. When would you use each?
- How did you ensure that your guessing game handled incorrect user input correctly?
- What strategies did you use to avoid infinite loops in your code? Provide an example where an infinite loop might occur.
- How does using loops make your multiplication quiz more efficient compared to manually writing each question?

Week 3: Lists, Functions, and Lambda Expressions

Task 1 (Simple) - Grocery List Manager

Ask the user to **enter grocery items**, store them in a list, and print the list when they are done.

Example Input:

```
Enter grocery item (or type 'done' to finish): Bread
Enter grocery item: Milk
Enter grocery item: Eggs
Enter grocery item: done
```

Desired Output:

```
Your grocery list: ['Bread', 'Milk', 'Eggs']
```

Task 2 (Intermediate) - Find the Biggest Number!
Write a function that takes a list of numbers and finds the **largest** one **without using** the `max()` function.

Example Input:

```
Enter numbers separated by space: 10 3 45 23 99 5
```

Desired Output:

```
The largest number is: 99
```

Task 3 (Advanced) - Mini-Encryption System
Create a function that replaces consecutive duplicate letters with the letter + its count.
For example, `"aaabbcc"` → `"a3b2c2"`.

Example Input:

```
Enter a string: aaabbcc
```

Desired Output:

```
Compressed string: a3b2c2
```

QUESTIONS WEEK 3

- Why are lists useful in Python? Provide an example where using a list simplified your code.
- Explain the importance of functions in programming. How did defining functions improve your code structure this week?
- How does a lambda function differ from a normal function? In what situations would you use a lambda function instead?
- What challenges did you face when working with functions that return values versus functions that print results directly?
- How did you debug your list-based tasks? Did you use print statements, list slicing, or any other techniques?

Week 4: Strings, Special Methods, and Regular Expressions

Task 1 (Simple) - Fun with Case Changing
Ask the user for a **sentence** and display:

1. All lowercase
2. All uppercase
3. Each word capitalized

Example Input:

```
Enter a sentence: I love Python!
```

Desired Output:

```
Lowercase: i love python!
Uppercase: I LOVE PYTHON!
Capitalized: I Love Python!
```

Task 2 (Intermediate) - Is It a Palindrome?
Write a function that **checks if a word is a palindrome** (same forward & backward).
Example: "racecar" is a palindrome.

Example Input:

```
Enter a word: racecar
```

Desired Output:

```
'racecar' is a palindrome.
```

Task 3 (Advanced) - Email Extractor
Write a **regex-based** program that finds all **email addresses** in a given text.

Example Input:

```
Enter text: Contact us at support@email.com and
info@website.com
```

Desired Output:

```
Extracted Emails: ['support@email.com',
'info@website.com']
```

QUESTIONS WEEK 4

- **Explain how string indexing and slicing work. Provide an example where slicing helped simplify your code.**
- **How do built-in string methods (e.g., `.lower()`, `.replace()`, `.split()`) make text processing easier? Give an example from your tasks.**
- **Describe how you used regular expressions this week. What was challenging about using the `re` module?**
- **What is the difference between handling strings manually and using Python's built-in functions? Which approach is better and why?**
- **How did you validate user input for a password checker? What additional security checks could you add?**

Week 5: File Handling and Data Structures

Task 1 (Simple) - Save Your Info
Write a program that **asks for the user's name and today's date**, then **saves it in a file**.

Example Output in `user_info.txt`:

```
Name: Alex
Date: 04/03/2025
```

Task 2 (Intermediate) - High Score Keeper
Write a program that **reads a file** containing game scores and prints the **highest score**.

Example `scores.txt`:

```
50
85
67
92
```

Desired Output:

```
The highest score is: 92
```

Task 3 (Advanced) - Word Frequency Counter **
Write a program that **reads a text file, counts how many times each word appears, and saves the result in a new file.

Example Output in `word_count.txt`:

```
Python: 5
is: 2
awesome: 3
```

QUESTIONS WEEK 5

- **How does file handling work in Python? Explain the difference between reading (`r`), writing (`w`), and appending (`a`) to a file.**
- **What are the advantages of using data structures like dictionaries, sets, and lists when processing file data?**
- **How did you ensure that your program properly closed files after reading/writing?**
- **Describe a situation where using a dictionary instead of a list would be more efficient.**
- **If you were working with a large dataset, what Python techniques would you use to optimize file reading and data storage?**

Final Notes About Submission

- Your submission should contain a Python file with all the completed tasks.
- You should also provide a short **10-minute video** answering each question using your tasks as examples.
- Ensure your code is well-commented and structured for clarity.

In [ ]: